Buggy code-

```
def stats_of_positives(nums):
   count = 0
   total = 0
   mx = 0
   for i in range(len(nums)+1):
      if nums[i] >= 0:
         count += 1
         total += nums[i]
         if nums[i] > mx:
            mx = nums[i]
   avg = total / count
   print("Average of positive numbers:", int(avg))
   print("Max value:", max)
data = [3, -1, 5, 0, 8]
stats_of_positives(data)
```

Prompt A- "Check this code for errors."

Output -

There are a few issues:

- The loop uses range(len(nums)+1) which will cause an IndexError.

- You're including zeros as positive numbers (>= 0).

- Dividing by count could be a division by zero if there are no positive numbers.

- Printing "max" is wrong; it should print mx.

- Casting the average to int will truncate.

Fixed code:

```
def stats_of_positives(nums):
```

```
positives = [x for x in nums if x > 0]
count = len(positives)
total = sum(positives)
if count == 0:
  print("Average of positive numbers: 0")
  print("Max value:", 0)
  return
avg = total / count
mx = max(positives)
print("Average of positive numbers:", avg)
print("Max value:", mx)
```

Prompt B- "Check this Python function for logical and semantic errors.
Goals: It should compute the average and maximum of strictly positive integers in the input list and print them.
Please do the following:

1. **Identify each bug and explain why it's a bug.**
2. **Provide a corrected version of the function that:**
    - **Uses > 0 (strictly positive)**
    - **Avoids IndexError and division-by-zero**
    - **Does not truncate the average**
    - **Prints the correct max variable**
3. **Keep the function name the same and keep the interface similar.**
4. **Add one short test snippet demonstrating correct output on the provided list."**

Output-

Bugs identified:

1) Off-by-one loop: range(len(nums)+1) leads to IndexError on the last iteration.

2) Wrong condition: >= 0 includes zeros, but the function name implies strictly positive numbers.

3) Division-by-zero: total / count fails if there are no positives.

4) Wrong variable printed: uses "max" instead of "mx".

5) Average is truncated: int(avg) discards the fractional part.

6) mx initialized to 0 can be misleading if there are no positives.

Corrected code (same function name, same interface):

```
def stats_of_positives(nums):
    count = 0
    total = 0.0
    mx = None
    for i in range(len(nums)):  # fixed off-by-one
        if nums[i] > 0:       # strictly positive
            count += 1
            total += nums[i]
            mx = nums[i] if (mx is None or nums[i] > mx) else mx

    if count == 0:
        print("Average of positive numbers: N/A (no positives)")
        print("Max value: N/A (no positives)")
        return

    avg = total / count
    print("Average of positive numbers:", avg)  # no truncation
    print("Max value:", mx)

# Test
data = [3, -1, 5, 0, 8]
stats_of_positives(data)
```

Pros

- **Clear bug list with reasoning.**
- **Satisfies the constraints (strictly positive, no truncation, no IndexError, handles no-positives).**
- **Includes a small test.**

Cons

- **No line numbers for precise mapping.**
- **Explanations are still brief; could include categories or severity.**

Prompt C- "You are a Python debugging assistant. Analyze the code below and respond only in the following structure and formatting:

Bugs

- **Bug #:**
    - **Line #: <line(s)>**
    - **Category: <Off-by-one | Wrong condition | Init error | Division-by-zero | Output formatting | Wrong variable reference | Other>**
    - **Explanation: <1–3 concise sentences>**
    - **Suggested Fix: <1–2 concise sentences>**

Output-

Bugs

- **Bug #: 1**
    - **Line #: 5**
    - **Category: Off-by-one**
    - **Explanation: range(len(nums)+1) iterates one index beyond the last valid index and will raise an IndexError.**
    - **Suggested Fix: Use range(len(nums)) or iterate directly over the list.**
- **Bug #: 2**

    - **Line #: 6**
    - **Category: Wrong condition**
    - **Explanation: The condition >= 0 includes zero, but the function is intended for strictly positive numbers.**
    - **Suggested Fix: Change to > 0.**
- **Bug #: 3**
    - **Line #: 11**
    - **Category: Division-by-zero**
    - **Explanation: If there are no positive numbers, count is 0 and total / count raises an error.**
    - **Suggested Fix: Check count == 0 and handle that case before dividing.**
- **Bug #: 4**
    - **Line #: 12**
    - **Category: Output formatting**
    - **Explanation: Casting to int truncates the average, changing the intended value.**
    - **Suggested Fix: Print the float directly or format to desired precision.**
- **Bug #: 5**
    - **Line #: 13**
    - **Category: Wrong variable reference**
    - **Explanation: Prints the built-in name max instead of the variable mx.**
    - **Suggested Fix: Print mx.**
- **Bug #: 6**
    - **Line #: 3**
    - **Category: Init error**
    - **Explanation: Initializing mx = 0 fails if there are no positives (the result would misleadingly remain 0).**
    - **Suggested Fix: Initialize mx = None and update only when a positive is found.**

Code-

```
def stats_of_positives(nums):
```

```
    count = 0
    total = 0.0
    mx = None
    for i in range(len(nums)):        # fixed off-by-one
        if nums[i] > 0:               # strictly positive
            count += 1
            total += nums[i]
            if mx is None or nums[i] > mx:
                mx = nums[i]
    if count == 0:                    # handle division-by-zero + no positives
        print("Average of positive numbers: N/A (no positives)")
        print("Max value: N/A (no positives)")
        return
    avg = total / count               # no truncation
    print("Average of positive numbers:", avg)
    print("Max value:", mx)
# brief test
data = [3, -1, 5, 0, 8]
stats_of_positives(data)
```

Pros

- **Highly structured and scannable.**
- **Includes line numbers, categories, and clear fixes.**
- **Produces a directly usable corrected code.**

Cons

- **Slightly longer response; requires the original code to be line-number consistent.**

Evaluation -

4) Evaluation Against Your Criteria

Criterion	Prompt A (Basic)	Prompt B (Detailed)	Prompt C (Structured)
Accuracy	**Catches most major bugs; may miss init nuance for mx**	**Catches all listed bugs with correct fixes**	**Catches all bugs with explicit mapping to lines and categories**
Clarity	**Medium: brief bullet list**	**High: clear bullet reasons + corrected code**	**Very high: strict sections, categories, and line references**
Actionability	**Good (provides fixed code)**	**Very good (fixed code + test)**	**Excellent (precise fixes, corrected code, and traceability)**
Structure	**Low: unstructured narrative**	**Medium: organized bullet list + code**	**Very high: enforced schema → easy to follow and implement**

Modified Code (intentionally leaving one subtle bug for testing)

```
def stats_of_positives(nums, include_zero=False):
    count = 0
    total = 0.0
    mx = None
    for x in nums:
        if (x > 0) or (include_zero and x == 0):
            count += 1
            total += x
            if mx is None or x > mx:
```

```
            mx = x
    if count == 0:
        print("Average of selected numbers: N/A (no values)")
        print("Max value: N/A (no values)")
        return
    avg = total / count
    print("Average of selected numbers:", avg)
    print("Max value:", mx)
# Quick checks
stats_of_positives([], include_zero=False)       # no values
stats_of_positives([-2, -5], include_zero=False)   # no values
stats_of_positives([0, 0.0], include_zero=True)    # zeros allowed
stats_of_positives([1.5, 2.5, 0], include_zero=False)
```